TP3: Quantum Phase Estimation

Foreword

Welcome to TP3. The previous TPs taught you to encode functions (TP1) and to search through them (TP2). This time, we are doing something subtler: we are extracting *numbers* hidden inside quantum operators. Specifically, given a unitary U and one of its eigenvectors $|\psi\rangle$, we want to learn the eigenvalue. Since U is unitary, that eigenvalue lives on the unit circle and looks like $e^{2\pi i\varphi}$ for some real $\varphi \in [0, 1)$. The whole game is to read φ off in binary, with as much precision as we have ancilla qubits to spare.

This algorithm, Quantum Phase Estimation (QPE), is the engine behind Shor's factoring algorithm, the HHL linear-solver, and basically every quantum chemistry simulation you will ever read about. It rests on two pieces: phase kickback (which you already know from TP1) and a new piece called the Quantum Fourier Transform (QFT). Section 1 builds the QFT, Section 2 plugs it into QPE, Section 3 explores the result in Python.

A few survival tips:

- **Endianness, the eternal nemesis.** Q# does not impose an endianness on raw `Qubit[]` arrays — that is purely an interpretation choice (the standard library even exposes `LittleEndian` and `BigEndian` types to make it explicit when needed). By convention in this TP, and in most of the standard library, `qs[0]` represents the least significant bit of an integer. Most QFT and QPE references on the web use the opposite (big-endian) convention, which is why the swap step at the end of the QFT exists. If your phase comes out reversed, swap the bits and breathe.
- **Phase kickback returns.** The trick from TP1, where putting the ancilla in the right state turns a controlled operation into a phase on the control register, is the entire conceptual heart of QPE. If you forgot, re-read TP1 Section 1.2 first.
- **Adjoint is your friend.** The inverse QFT is just the QFT run backward. Q#'s `Adjoint` keyword does this for free, provided your QFT is built from adjointable operations. No need to write a second function.
- **Watch out for $4/\pi^2$.** You will read that QPE succeeds with probability "at least $4/\pi^2 \approx 40\%$". That is the worst case, when the phase falls exactly between two representable values. For typical phases, success rates are much higher. Do not be alarmed if your simulator hits 100% on toy examples.
- **`Fact` is your friend for sanity checks.** Q#'s standard library provides `Fact(condition, message)`, which fails loudly with your message if the condition is false. It is the cleanest way to assert preconditions on inputs (`Length(qs) > 0`, `nbAncillas >= 1`, etc.) without polluting the logic with manual `if/fail` blocks. You are encouraged to sprinkle them through your code where assumptions matter.
- **`Fact` is allowed.** Q#'s `Fact(condition, message)` from `Std.Diagnostics` is a runtime assertion: it crashes loudly if the condition is false, and does nothing otherwise.

Sprinkle it through your code to catch dimension mismatches and convention errors early. `Fact(Length(qs) > 0, "register must be non-empty")` is the kind of one-liner that saves an hour of debugging.

Each exercise comes with a **Hint** box to unblock you. Appendix A explains *why* QPE works, with the same friendly tone as the Grover appendix from TP2.

1 Quantum Fourier Transform (QFT)

These exercises are found in the `QFT.qs` file.

The Quantum Fourier Transform is a quantum analog of the classical discrete Fourier transform. It is a fundamental building block for many quantum algorithms, including quantum phase estimation, Shor's algorithm, and quantum algorithms for solving systems of linear equations.

1.1 Theoretical Background

For a quantum state $|x\rangle$ where $x \in \{0, 1, \ldots, N-1\}$ and $N = 2^n$, the QFT is defined as:

$$\mathrm{QFT}\,|x\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i x y/N}\,|y\rangle \tag{1}$$

This sum form is the cleanest *definition*, but it is useless for building a circuit. The miracle of the QFT is that the result of equation (1) factorizes as a tensor product of single-qubit states. Writing x in binary as $x_1 x_2 \ldots x_n$ (with x_1 the most significant bit), one can show:

$$\mathrm{QFT}\,|x_1 x_2 \ldots x_n\rangle = \frac{1}{\sqrt{2^n}} \big(\,|0\rangle + e^{2\pi i\,[0.x_n]}\,|1\rangle\,\big) \otimes \big(\,|0\rangle + e^{2\pi i\,[0.x_{n-1}x_n]}\,|1\rangle\,\big) \otimes \cdots \otimes \big(\,|0\rangle + e^{2\pi i\,[0.x_1 x_2 \ldots x_n]}\,|1\rangle\,\big) \tag{2}$$

where $[0.x_j x_{j+1} \ldots x_n]$ denotes the binary fraction:

$$[0.x_j x_{j+1} \ldots x_n] = \frac{x_j}{2} + \frac{x_{j+1}}{4} + \ldots + \frac{x_n}{2^{n-j+1}} \tag{3}$$

Equation (2) is the *effective* action of the QFT, the form you can derive by direct computation from equation (1). However, the natural sequence of Hadamards and controlled rotations described in Section 1.3 produces the *same product but with the qubits in reverse order*:

$$\text{(circuit output, before swap)} = \frac{1}{\sqrt{2^n}} \bigotimes_{j=1}^{n} \Big(|0\rangle + e^{2\pi i\,[0.x_j x_{j+1} \ldots x_n]}\,|1\rangle\Big) \tag{4}$$

The two are reconciled by the final swap pass at the end of the circuit, which inverts the qubit order. This is also where the endianness mismatch between most QFT references (big-endian, as in equation (2)) and Q#'s little-endian convention crystallizes.

1.2 Controlled Rotation Gates

Before implementing the QFT, we need controlled rotation gates. The QFT circuit uses controlled R_k gates where:

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{i2\pi/2^k} \end{pmatrix}$$

In Q#, the `R1` gate applies a phase $e^{i\theta}$ to the $|1\rangle$ component:

$$R1(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

So R_k is exactly `R1(2.0 * PI() / IntAsDouble(1 <<< k))`, and you can make it controlled with the `Controlled` keyword.

Hint. The angle of R_k is a function of k. Be careful with types: `R1` expects a `Double`, while 2^k is naturally an `Int`. The standard library has helpers (`IntAsDouble`, `1 <<< k`) to bridge the gap cleanly.

A worked example to anchor your intuition: between two adjacent qubits in the QFT, the angle should be $\pi/2$. Two qubits further, $\pi/8$. The further apart the qubits, the smaller the angle, the gentler the correction.

1.3 QFT Circuit Implementation

The QFT circuit follows this pattern. We use 1-indexed qubit positions in the description below; remember that Q# is 0-indexed, so adjust the loop bounds when writing code.

For each qubit j from 1 to n:

- Apply a Hadamard gate to qubit j.
- For each qubit k from $j + 1$ to n:
  - Apply a controlled R_{k-j+1} gate, with qubit k as control and qubit j as target.

Then apply swap operations to reverse the order of the qubits: swap qubit 1 with n, qubit 2 with $n - 1$, and so on. This step is what reconciles equations (2) and (4) above.

Implement the `QFT(qs: Qubit[]): Unit is Adj` operation. Make sure it is declared `is Adj` (or `is Adj + Ctl` if you want maximum reusability) so that you can later get the inverse QFT for free with `Adjoint QFT`.

A word on complexity. The circuit above uses n Hadamard gates and $n(n-1)/2$ controlled rotations, so $O(n^2)$ gates total. Compare to the classical Fast Fourier Transform, which runs in $O(N \log N) = O(n\, 2^n)$ on inputs of size $N = 2^n$. The QFT is exponentially faster than the FFT in terms of gate count, which is part of what makes algorithms like Shor's factoring possible. The catch, of course, is that the QFT acts on quantum amplitudes rather than classical numbers, so you cannot simply "read off" the Fourier coefficients at the end — but for QPE and friends, that is exactly what we need.

Hint. Two nested loops, then a final swap pass. The outer loop picks the target qubit, the inner loop walks through all the qubits to its right and applies one controlled rotation per pair. The angle of each rotation depends only on the *distance* between the two qubits, not on their absolute positions.

For the swap pass, you can write a small loop that pairs `qs[i]` with `qs[n-1-i]`, or just call `SwapReverseRegister(qs)` from the standard library, which does exactly that in one line.

Watch the loop bounds: 0-indexed Q# arrays mean the description "qubit j from 1 to n" translates to `for j in 0..n-1`, and "qubit k from $j + 1$ to n" becomes `for k in j+1..n-1`.

2 Quantum Phase Estimation (QPE)

These exercises are found in the `QPE.qs` file.

Quantum Phase Estimation estimates the phase (eigenvalue) of an eigenvector of a unitary operator. It is one of the most important quantum algorithms and serves as a subroutine in many others.

Convention used in this section. The ancilla register is treated as little-endian, consistent with Q#'s standard library: `ancillas[0]` carries the *least* significant bit of the encoded phase, and `ancillas[k]` controls the gate U^{2^k}. After the inverse QFT, reading the register as a little-endian integer and dividing by 2^n gives $\tilde{\varphi}$ directly. The pedagogical Appendix A uses the opposite (big-endian) convention because most textbook derivations do; do not be surprised if the indices look swapped there.

2.1 Problem Statement

Given a unitary operator U and one of its eigenvectors $|\psi\rangle$ such that:

$$U|\psi\rangle = e^{2\pi i\varphi}|\psi\rangle$$

where $\varphi \in [0, 1)$ is the unknown phase, the goal is to find an approximation of φ with high probability.

The algorithm uses n ancilla qubits to estimate φ to n bits of precision. If you want to understand *why* this works (the role of phase kickback, the link with QFT, and where the precision bound comes from), read Appendix A before continuing.

2.2 QPE Circuit

The algorithm has three main steps:

- **Initialization**: Prepare n ancilla qubits in the state $|+\rangle^{\otimes n} = H^{\otimes n}|0\rangle^{\otimes n}$, and the eigenstate register in $|\psi\rangle$.
- **Controlled Unitary Operations**: Apply controlled powers of U to the eigenstate register. With Q#'s little-endian ancilla convention:
  - C-U^{2^0} controlled by `ancillas[0]` (the least significant bit)
  - C-U^{2^1} controlled by `ancillas[1]`
  - ...
  - C-$U^{2^{n-1}}$ controlled by `ancillas[n-1]` (the most significant bit)
- **Inverse QFT**: Apply the inverse QFT to the ancilla register and measure it.

Hint. Three blocks, in order: prepare ancillas in superposition, apply controlled powers of U with each ancilla as a control, then undo with the inverse QFT.
For applying U^{2^k}, a small helper that calls U a number of times in a loop is the simplest path. Think about how to make that helper `Ctl`-able so you can wrap a `Controlled` around it.
The inverse QFT is just `Adjoint QFT`, no extra work needed if your QFT is declared `is Adj`.

2.3 The Single-Qubit Test Case (`QPEPhi`)

For this exercise, you will implement `QPEPhi(phi: Double, nbAncillas: Int): Result[]` in `QPEPlus.qs`. The phase φ is given as an argument so that the test harness can sweep over different values.

The eigenvalue equation we want to satisfy is:

$$U\,|1\rangle = e^{2\pi i\varphi}\,|1\rangle$$

Your job is to figure out which standard single-qubit gate U has $|1\rangle$ as eigenvector with eigenvalue $e^{2\pi i\varphi}$.

Hint. You need a single-qubit gate U such that $|1\rangle$ is an eigenvector with eigenvalue $e^{2\pi i\varphi}$. Among the gates you have already used, look for one that takes an angle as a parameter and applies a phase to $|1\rangle$ while leaving $|0\rangle$ untouched.
Preparing $|1\rangle$ from $|0\rangle$ is a one-line operation. Then your QPE call gets the controlled version of U as its first argument.

3 Python Analysis

The accuracy of quantum phase estimation depends on the number of ancilla qubits used. With n ancilla qubits, the algorithm can distinguish phases that differ by at least 2^{-n}.

Theoretical Analysis

QPE estimates the phase φ by measuring the ancilla qubits after the inverse QFT. The measurement gives a binary string that represents an approximation of $\varphi \cdot 2^n$.

If the true phase is φ and we measure the binary string $b_{n-1}b_{n-2}\ldots b_1b_0$, then the estimated phase is:

$$\tilde{\varphi} = \frac{\text{measured integer}}{2^n} = \frac{\sum_{j=0}^{n-1} b_j \cdot 2^j}{2^n}$$

Accuracy and Error Analysis

QPE returns the closest n-bit approximation $\tilde{\varphi} = \lfloor 2^n\varphi \rceil / 2^n$ with probability at least $4/\pi^2 \approx 0.405$ (this is the worst case, when φ falls exactly halfway between two representable values). When QPE returns this best approximation, the error is bounded by:

$$|\varphi - \tilde{\varphi}| \leq 2^{-n-1}$$

since the closest n-bit fraction is at most half a step away from φ, and one step is 2^{-n}.

A few useful refinements:

- If φ *can* be represented exactly with n bits, QPE returns the exact value with probability 1.
- For typical φ (not pathologically halfway between two grid points), the success probability is much higher than $4/\pi^2$, often above 80%.
- Failure modes return one of the neighbours of the best approximation, so even when QPE "fails" the error is usually still close to 2^{-n}.

- Resolution improves *exponentially* with the number of ancillas: each extra ancilla doubles the precision and roughly halves the error.

Your task

In Python, convert the result of `QPEPhi` to a floating-point number using the formula above, and observe how it gets closer to φ as you increase the number of ancilla qubits.

Hint. The `qsharp` package lets you import Q# operations as Python callables:

```
import numpy as np
import qsharp
qsharp.init(project_root='.')
from qsharp.code.QPEPlus import QPEPhi
```

From there, `QPEPhi(phi, nbAncillas)` returns a list of `Result`s. Wrap the bit-to-float conversion in a small helper, something like `decode_phi(result_list)`, that takes the list of measurement outcomes and returns the corresponding $\tilde{\varphi} \in [0, 1)$. Two reasons:

- You will call it dozens of times in your sweeps and plots; one bug fixed in one place is much better than the same bug copy-pasted everywhere.
- It keeps the bit-order question in one well-defined function, making the convention easy to flip if your first attempt is reversed.

The conversion itself is a small loop: each bit contributes a power of 2^{-1}, and you need to be careful about which end of the array carries the most significant bit. If `decode_phi` consistently lands on $1 - \varphi$ instead of φ, or off by a power of two, try reversing the list inside the helper and re-run.

For the convergence plot, fix a phase like $\varphi = 1/3$, sweep `nbAncillas` from 2 to 10, run a few hundred shots per value, and plot the error. You should see it shrink roughly as 2^{-n}.

A How QPE actually works (the friendly version)

Quantum Phase Estimation looks intimidating because it combines two unfamiliar pieces: phase kickback and the QFT. But once you see it the right way, it is just a clever way of writing a number in binary, one bit at a time, using interference. This appendix walks through the mechanics with as little algebra as possible.

Convention used here. To match Section 2 (and Q#'s standard library), we treat the ancilla register as little-endian: ancilla index k holds the bit weighted by 2^k, and ancilla k controls U^{2^k}. Most textbook derivations use the opposite convention; if you read elsewhere, just flip the indices in your head.

Step 1: what we are trying to read

We have a unitary U and one of its eigenvectors $|\psi\rangle$. The eigenvalue is some complex number on the unit circle, and we write it as $e^{2\pi i\varphi}$ for some $\varphi \in [0, 1)$. Why this awkward parametrization? Because it makes the bookkeeping clean: applying U once adds the phase $e^{2\pi i\varphi}$, applying U twice adds $e^{2\pi i \cdot 2\varphi}$, applying U^{2^k} adds $e^{2\pi i \cdot 2^k \varphi}$. The factors of 2 pile up nicely, which is exactly what we want for a binary representation.

Our goal: extract the bits of φ when written in binary, $\varphi = 0.\varphi_1\varphi_2\varphi_3\ldots$ (base 2). With n ancilla qubits, we will recover the first n bits.

Step 2: phase kickback, refreshed

Recall from TP1 that if $|\psi\rangle$ is an eigenvector of U with eigenvalue $e^{2\pi i\varphi}$, then a controlled-U acting on $|c\rangle|\psi\rangle$ (where c is a single control qubit) does:

$$C\text{-}U\ |c\rangle|\psi\rangle = e^{2\pi i\varphi\cdot c}|c\rangle|\psi\rangle$$

The phase appears on the control qubit and *not* on $|\psi\rangle$, which stays in its eigenstate the whole time. This is the same trick as TP1, generalized: instead of $(-1)^{f(x)}$, we get $e^{2\pi i\varphi}$.

If we apply $C\text{-}U^{2^k}$ instead, the phase is $e^{2\pi i\cdot 2^k\varphi}$. So if we have n ancillas in superposition and we apply $C\text{-}U^{2^0}, C\text{-}U^{2^1}, \ldots, C\text{-}U^{2^{n-1}}$ each controlled by a different ancilla, each ancilla gets its own "slice" of the phase.

Step 3: stacking phases on the ancillas

Start with n ancillas in $|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}}\sum_x |x\rangle$ (all bit strings in superposition) and the target register in $|\psi\rangle$.

Each ancilla qubit k controls one application of U^{2^k}. By phase kickback, that contributes a factor $e^{2\pi i\cdot 2^k\varphi}$ when ancilla k is in $|1\rangle$, and nothing when it is in $|0\rangle$. Writing the ancilla register as a binary string $x = x_0x_1\ldots x_{n-1}$ (where $x_k \in \{0,1\}$ is the bit on ancilla k), the total phase picked up on the basis state $|x\rangle$ is:

$$\prod_{k=0}^{n-1} e^{2\pi i\, x_k\cdot 2^k\varphi} \;=\; e^{2\pi i\,\varphi\sum_{k=0}^{n-1} x_k\cdot 2^k} \;=\; e^{2\pi i\,\varphi\, x}$$

where in the last step we used the fact that $\sum_k x_k\cdot 2^k$ is just the integer x written in binary.

After all the controlled operations, the ancilla register is therefore in:

$$\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1} e^{2\pi i\, x\,\varphi}|x\rangle$$

Pause and stare at this state. Ignoring the $\sqrt{2^n}$ normalization, the amplitude of $|x\rangle$ is $e^{2\pi i\, x\varphi}$. This is exactly what comes out of a QFT applied to the basis state $|2^n\varphi\rangle$, if $2^n\varphi$ happened to be an integer. **Phase kickback has secretly written $2^n\varphi$ into the Fourier-transformed ancilla register**, all without us touching the ancillas in any complicated way.

Step 4: the inverse QFT reads the integer

If $2^n\varphi$ is an integer, the state above is exactly $\mathrm{QFT}\,|2^n\varphi\rangle$. Applying QFT^{-1} (the inverse QFT) brings us back to $|2^n\varphi\rangle$, and measuring in the computational basis gives us $2^n\varphi$ as a binary number. Dividing by 2^n gives φ. **Done.**

When $2^n\varphi$ is *not* an integer (the realistic case), the inverse QFT does not produce a sharp basis state. Instead, the amplitude is concentrated around the closest integer to $2^n\varphi$, with smaller amplitudes spreading out to neighbors. Measuring still gives the closest integer with high probability; that probability is at least $4/\pi^2 \approx 40\%$ in the absolute worst case (phase exactly between two integers) and typically much higher.

Step 5: where the bound 2^{-n} comes from

If we read n bits of φ, the smallest difference we can resolve is 2^{-n} (the distance between two consecutive n-bit fractions). So the estimation error $|\varphi - \tilde{\varphi}|$ is at most 2^{-n} when QPE succeeds. Want twice the precision? Add one ancilla. Want one more decimal digit? Add about 3.32 ancillas (since $\log_2 10 \approx 3.32$).

This is the same exponential resolution you get from binary representations of real numbers, surfaced through quantum interference. Nothing more, nothing less.

Step 6: the takeaway

QPE writes the eigenvalue's phase into the ancilla register via phase kickback, then the inverse QFT translates "phase" into "binary number". Read off the bits.

Every line of the QPE circuit serves one of those two purposes: either it is part of the phase-kickback pile-up (the controlled U^{2^k} gates), or it is part of the readout (the inverse QFT). The Hadamards at the start are just preparing the uniform superposition that lets the kickback "write" across all n bits in parallel.